Sparse Index Tracking on the S&P 500

Replicating a 500-stock index with a handful of assets: three methods, and what happens once trading costs are real

Machine Learning in Finance — Project Report

1. The Problem

Owning the entire S&P 500 means holding 500 positions and rebalancing all of them on every adjustment. For a smaller portfolio this is costly and impractical. **Sparse index tracking** asks whether a much smaller basket — ten, thirty, or fifty stocks — can be chosen and weighted so that it rises and falls almost in step with the full index. Fewer holdings are cheaper and simpler to manage, but harder to keep aligned with the benchmark. This report studies where that balance lands, comparing three methods of building such a basket and then asking the question that decides everything in practice: which method survives once the cost of trading is taken into account?

2. Methods

The project implements three approaches, sharing one common testing framework so the comparison is fair.

Market-cap Top-K (baseline)

Take the K largest companies in the index and weight them by size. There is no optimization here — it is the obvious, naive choice, and serves as the benchmark any cleverer method should beat. Its holdings change slowly, since company sizes drift gradually.

LASSO with cardinality control

A machine-learning method that examines how stocks have moved over the past year and selects a small group whose combined behavior best matches the index. It works by penalizing the use of too many stocks, automatically pushing unhelpful ones to zero weight, with a search routine that tunes the penalty until the basket reaches the desired size. A small technical wrinkle: under the requirement that all weights be positive and sum to one, the standard penalty does nothing, so the method is split into two steps — first choosing *which* stocks to hold, then deciding *how much* of each.

SLAIT (Benidis, Feng & Palomar, 2018)

A specialized algorithm that solves the tracking problem directly, treating the number of holdings as a hard limit rather than something coaxed out by a penalty. It uses the original authors' published R package, called from Python, so the results match the method exactly as designed.

> **How the methods are tested.** Every method is evaluated with a *walk-forward* backtest: at each monthly rebalance the model is retrained on the most recent year of weekly data and held for the following month, rolling forward across a decade. The data is divided into an *in-sample* portion (used to develop and tune the approach) and an *out-of-sample* portion (kept completely untouched until the final test). The setup avoids common backtesting traps — it uses only information available at each point in time, and it includes companies that later left the index, so the results are not flattered by hindsight.

3. Summary Statistics

The table below reports out-of-sample results at a realistic trading cost of 10 basis points per trade, for the cardinalities shared across methods. *Tracking error* is the headline metric: it measures how far the portfolio strays from the index, so lower is better. CAGR is the annual growth rate, and turnover measures how much trading the method requires.

Method	K	Tracking error	Annual return	Volatility	Turnover
Top-K	10	12.2%	42.0%	22.6%	106%
Top-K	50	5.2%	32.1%	16.8%	93%
LASSO	10	19.4%	10.5%	29.6%	1244%
LASSO	50	5.0%	20.7%	15.6%	1616%
SLAIT	10	7.2%	13.1%	16.0%	1500%
SLAIT	50	4.4%	19.3%	14.6%	1459%
Benchmark	500	0.0%	21.2%	14.4%	—

Out-of-sample period (2023–2026, 39 monthly rebalances), 10 bps trading cost. Benchmark is the S&P 500 Total Return index.

Two clear patterns emerge. First, tracking error falls as the basket grows — more stocks allow closer replication. Second, at the same number of holdings, **SLAIT tracks the index most closely**: with only ten stocks it stays far tighter to the benchmark than the other two. LASSO struggles badly with very small baskets but becomes competitive once it is allowed more stocks. Top-K, despite its simplicity, tracks reasonably and improves steadily as K grows.

4. Does It Generalize?

A backtest is only trustworthy if its performance holds up on data that played no part in building the method. Comparing the development period with the held-out period, tracking error stayed remarkably stable for every method — the differences were mostly a fraction of a percentage point. This means the methods genuinely learned to track the index rather than memorizing the past. The ranking held too: SLAIT remained the tightest tracker in both periods, confirming that its edge is real and not an artifact of tuning. The rolling retraining is what makes this work — each month the model adapts to the latest market conditions, so it copes with new regimes such as the 2023 banking stress and the 2023–2024 rally.

5. Turnover and Transaction Costs

This is where the methods diverge most sharply, and where the project's central lesson lies. The turnover figures in the summary table differ by more than tenfold: Top-K trades modestly, while LASSO and SLAIT reshuffle their holdings aggressively at every rebalance. The two optimization methods chase the best-fitting basket each month, which means constantly buying and selling; the simple baseline mostly re-ranks the same large companies.

That trading is not free. The chart below summarizes how net return responds as the trading cost rises from zero to twenty basis points:

Method (K=50)	Return at 0 bps	Return at 20 bps	Lost to costs
Top-K	32.2%	31.9%	0.3 pts
LASSO	22.7%	18.8%	3.9 pts
SLAIT	21.0%	17.5%	3.5 pts

Net annual return at increasing trading costs, out-of-sample, K=50.

The simple baseline barely notices the cost, losing a fraction of a percentage point. The two optimizers lose around ten times as much. The consequence is striking: at zero cost, LASSO and SLAIT start *above* the benchmark, but as costs rise they fall *below* it, crossing over at a cost level well within the realistic range. In other words, the advantage the optimizers appear to have when trading is treated as free simply evaporates once it is priced in. A method that looks excellent on paper can be expensive to actually run — and a stable, low-turnover rule can quietly win on a net basis despite tracking less tightly.

6. A Word of Caution on "Beating" the Index

Over the test period, several of the portfolios outperformed the S&P 500, with the concentrated ten-stock Top-K basket returning roughly double the index. It is tempting to read this as success, but for an index tracker it is the opposite. The goal is to *match* the benchmark, not to bet against it. That same Top-K basket carries the highest tracking error of any method — it deviates from the index the most, and over this particular rally-driven stretch the deviation happened to point upward, fueled by its heavy tilt toward a few large technology names. On a different stretch the same bet could have gone the other way. The portfolios that genuinely fulfill the tracking objective are the ones that sit closest to the benchmark in both risk and return — SLAIT and LASSO at larger baskets — not the ones that posted the flashiest returns.

7. Conclusion

All three methods can replicate the S&P 500 with a small fraction of its constituents, and all improve as the basket grows. SLAIT is the most accurate tracker, especially with few holdings; LASSO needs a reasonable number of stocks to work well; Top-K is a stable, cheap baseline that is hard to dislodge. But the decisive finding is about cost, not accuracy: the high-turnover optimizers pay heavily for their precision, and once realistic trading costs are applied, a simple market-cap rule becomes genuinely competitive. Tracking accuracy and net-of-cost performance are driven by different forces, and judging a method on accuracy alone — or worse, on raw returns — can point to the wrong choice. The practical takeaway is that the best method depends as much on how much it costs to trade as on how well it tracks.

Tools: Python (pandas, NumPy, CVXPY) and R (sparseIndexTracking via rpy2). Data: Yahoo Finance prices and a public point-in-time S&P 500 constituents dataset. Full code and notebooks are available in the project repository. AI assistance (Claude, by Anthropic) was used for code drafting, methodology discussion, and editing under the author's direction; all decisions and verification are the author's own.